

Dev Team — Manual de Usuario

Version del plugin: 2.6.x · **Agentes:** 15 · **Comandos:** 22

Dev Team es un equipo completo de desarrollo con agentes IA que cubre todo el ciclo de vida del software: Historia de Usuario → diseño UI/UX → arquitectura → desarrollo → QA con evidencia → seguridad → pases a ambientes → deploy → documentacion. Incluye una **wiki viva** del proyecto, una **oficina virtual en vivo** para ver al equipo trabajar y **metricas de productividad y consumo**.

Funciona con proyectos **nuevos o existentes**, **mono-repo o multi-repo**, con backlog en **GitHub** o **Azure DevOps**.

Indice

Primeros pasos

1. [Instalacion](#)
2. [Inicio rapido](#)
3. [Conceptos que conviene conocer](#)
4. [El equipo: 15 agentes](#)
5. [Las reglas del equipo](#)
6. [Referencia de comandos](#)

Casos de uso — con ejemplo concreto de principio a fin

7. [Caso 1: Crear un proyecto nuevo desde una idea](#)
8. [Caso 2: Crear un proyecto desde un documento de requerimientos](#)
9. [Caso 3: Retomar un proyecto existente \(onboard\)](#)
10. [Caso 4: Una feature de principio a fin](#)
11. [Caso 5: Diseñar pantallas con mockups antes de programar](#)
12. [Caso 6: Reportar y corregir un bug](#)
13. [Caso 7: Debug de un problema que cruza VARIOS repos](#)
14. [Caso 8: Pruebas — plan, E2E, exploratoria y regresion](#)
15. [Caso 9: Base de datos — health, comparacion y scripts](#)
16. [Caso 10: Pase a un ambiente \(release\)](#)
17. [Caso 11: Auditoria de seguridad](#)
18. [Caso 12: La wiki del proyecto](#)
19. [Caso 13: Ver al equipo trabajar](#)
20. [Caso 14: Trabajar un sprint completo con Scrum](#)
21. [Caso 15: Ajustar los modelos de los agentes](#)

Referencia

22. [Configuracion del proyecto](#)
23. [Como cuidar tu limite de uso](#)
24. [Solucion de problemas](#)
25. [Preguntas frecuentes](#)

1. Instalacion

```
# 1. Agregar el marketplace (una sola vez)
/plugin marketplace add faast-app/faast-claude-marketplace

# 2. Instalar el plugin
/plugin install dev-team@faast-marketplace

# 3. Actualizar cuando haya cambios publicados
claude plugin marketplace update faast-marketplace
claude plugin update dev-team@faast-marketplace
```

```
# → reinicia la sesion de Claude Code despues de actualizar
# (agentes, hooks y comandos se cargan al inicio de sesion)
```

Requisitos (el agente **setup** los valida e instala por ti la primera vez): git · Docker · Node.js ≥ 18 · **gh** (GitHub) o **az** (Azure DevOps) · cliente de BD (mysql/sqlcmd/psql) · Playwright.

2. Inicio rapido

No necesitas memorizar comandos. Abre Claude Code **en la carpeta del proyecto** (o donde quieras crearlo) y escribe:

```
/dev-team:start
```

El equipo detecta tu situacion:

Situacion	Que pasa
No hay proyecto configurado	Te ofrece crear uno nuevo o heredar uno existente
Hay proyecto	Te muestra un resumen de 5 lineas y te propone la siguiente accion
Pediste algo especifico	/dev-team:start hay un bug en el login → enruta al flujo correcto

Tip: para preguntas simples ("¿que estados tiene una solicitud?") pregunta directo, sin **/start** — el router puede disparar agentes que no necesitas.

3. Conceptos que conviene conocer

.coordination/ — la carpeta de coordinacion del equipo. Vive en la raiz del mono-repo o en la carpeta paraguas del multi-repo. Contiene el config, los handoffs entre agentes, la wiki, la evidencia de QA, las metricas y los pases. Es la memoria compartida del equipo.

Topologias — **mono** (un repo, servicios como carpetas) o **multi** (carpeta paraguas con un repo git por servicio). El equipo se adapta a la tuya y **nunca** propone migrarla.

Handoffs — los agentes se comunican SOLO por archivos markdown en **.coordination/handoffs/** (**{de}-a-{para}-{fecha}.md**). Todo queda trazable.

Gates — nada avanza sin pasar sus puertas: QA aprueba antes del merge, cybersec audita lo sensible, el release-manager audita los pases, y **solo el Lead mergea**.

Plan primero — antes de ejecutar features o fixes, el Lead te presenta el plan y espera tu confirmacion. Tu decides; el equipo ejecuta.

Wiki viva — [.coordination/wiki/](#) es la memoria destilada del proyecto (patron LLM Wiki). Los agentes la leen antes de cada tarea; solo el tech-writer la escribe. Abrela con Obsidian para ver el grafo de conocimiento.

4. El equipo: 15 agentes

Agente	Rol	Modelo
 setup	Valida e instala prerequisites, configura conexiones BD y tracker	haiku (fijo)
 product-owner	HUs y bugs en lenguaje 100% de negocio; backlog Scrum en GitHub/Azure	sonnet
 architect	Arquitectura: topologia, servicios, stack, BD, gateway	sonnet (opus opcional)
 ui-designer	Mockups y specs UI/UX: paletas WCAG, tipografia, estados	sonnet
 lead	Coordina, presenta planes, asigna, exige gates, unico que mergea	sonnet
 backend	Servicios backend (.NET 8, Node.js, Python, Java)	sonnet
 frontend	SPA y microfrontends (React, Vue, Angular)	sonnet
 dba	Esquemas, migraciones, scripts de pase, comparacion de BDs	sonnet
 qa (QA Lead)	Plan de pruebas, reparto a especialistas, veredicto, suite E2E	sonnet
 qa-frontend	QA de UI: Playwright interactivo, responsive, evidencia visual	sonnet
 qa-backend	QA de APIs: contract testing, permisos, casos borde	sonnet
 release-manager	Solicitud de pase (Word+PDF), audita scripts del DBA, Scripts.zip	sonnet
 infra	Docker, CI/CD, gateways, deploy + informe de conformidad	sonnet
 cybersec	Auditoria de seguridad; nunca commitea	sonnet
 tech-writer	Docs tecnicas + mantenedor de la wiki	haiku (fijo)

Modelos: optimizados para costo — nadie usa opus por defecto y **fable** esta prohibido.
Configurables por proyecto o por usuario (ver [Caso 15](#)); los de haiku son fijos.

5. Las reglas del equipo (siempre activas)

1. **PLAN PRIMERO** — antes de ejecutar una feature o un fix, el Lead presenta el plan (que/quien/donde/riesgos) y espera tu confirmacion. Puedes ajustar o pedir otro abordaje. Nada se ejecuta sin tu OK.
2. **El PO redacta TODO en lenguaje funcional de negocio** — HUs, bugs e items que entiende cualquier persona no programadora. Titulos limpios, sin codigos raros. Lo tecnico va al final o en los handoffs.
3. **Se trabaja con Scrum** — sprints con Sprint Goal, story points (1-8), backlog por valor, refinamiento antes del planning, review y retrospectiva.
4. **QA es un equipo y es gate de merge** — nada llega a main sin veredicto APROBADA con evidencia.
5. **REGLA DE ORO (fija e inalterable): QA no valida sin informe de conformidad** — en ambientes desplegados exige el informe (version exacta, alcance, health); en desa exige el stack COMPLETO levantado. Nadie puede saltarse este flujo.
6. **LEY QA: a la primera falla, reporta** — evidencia + **blocked** + reporte inmediato. Prohibido reintentar, workarounds o probar fuera de su alcance.
7. **QA no debuggea** — reproduce, documenta y reporta. La causa raiz es del dev.
8. **Evidencia SIEMPRE embebida** — screenshots/clips visibles DENTRO del item: GitHub → rama **evidence** + **![] (raw)**; Azure → attachment + **** en el HTML del WI. Jamas un link suelto.
9. **Cybersec es segundo gate** en auth/datos sensibles y nunca commitea.
10. **Solo el Lead mergea**; un agente = un branch = una tarea.
11. **El release-manager es gate de pases** — audita el paquete RESULTANTE y puede rechazar al DBA hasta que cumpla el formato global.
12. **Los devs preguntan antes de abrir PR** — no todo entregable lleva PR.
13. **Solo el Lead delega en subagentes** (puede paralelizar: 2 backend en HUs distintas; nunca otro lead). Los demas ejecutan directo — un hook bloquea la delegacion anidada. Excepcion universal: Explore (busqueda barata).
14. **Nada de personas ni valores hardcoded** — nombres, correos, reviewers, rutas: siempre del config o preguntando.
15. **Todo queda registrado automaticamente** — hooks del plugin escriben la actividad en **metrics/activity.jsonl** (alimenta oficina y metricas).

6. Referencia de comandos

Diario

Comando	Que hace
/dev-team:start	Punto de entrada universal: detecta contexto y te guia
/dev-team:status	Estado del proyecto: sprint, backlog, handoffs, bloqueos
/dev-team:sync	Sincroniza con GitHub/Azure: trae tickets, sube avances, crea PRs
/dev-team:inbox	Un agente lee sus tareas y handoffs pendientes

Proyectos

Comando	Que hace
<code>/dev-team:new-project {doc o idea}</code>	Proyecto nuevo: arquitectura → repos → backlog → wiki
<code>/dev-team:onboard {nombre}</code>	Hereda un proyecto existente
<code>/dev-team:setup</code>	Valida/instala prerequisites (<code>setup db</code> , <code>setup tracker</code> , <code>setup playwright</code>)

Trabajo

Comando	Que hace
<code>/dev-team:refine {pedido}</code>	El PO convierte tu pedido en HUs en el tracker
<code>/dev-team:assign-task</code>	El Lead asigna trabajo (con plan primero)
<code>/dev-team:handoff</code>	Crear comunicacion entre agentes

Calidad

Comando	Que hace
<code>/dev-team:test-plan {HU}</code>	Plan de pruebas desde los criterios de aceptacion
<code>/dev-team:e2e {HU run explorar url}</code>	Validacion Playwright + suite E2E automatizada
<code>/dev-team:review-pr {n}</code>	Revision de codigo de un PR
<code>/dev-team:security-audit</code>	Auditoria de seguridad del repo actual
<code>/dev-team:git-check</code>	Verificacion git antes de commitear

Operacion y releases

Comando	Que hace
<code>/dev-team:db-health</code>	Health check de BD (esquema, indices, slow queries)
<code>/dev-team:deploy-check</code>	Readiness para deploy
<code>/dev-team:pase {ambiente}</code>	Solicitud de pase completa: doc Word+PDF + Scripts.zip auditado
<code>/dev-team:document {tema}</code>	Tech-writer actualiza documentacion tecnica

Conocimiento y visibilidad

Comando	Que hace
<code>/dev-team:wiki {init ingest lint query}</code>	Wiki viva del proyecto (vault de Obsidian)
<code>/dev-team:team-office</code>	Oficina virtual 2D en vivo
<code>/dev-team:team-metrics</code>	Productividad y consumo de tokens por agente

Los comandos se ejecutan **inline** en tu sesion (no como subagentes) — el plugin lo garantiza con un hook. Solo los AGENTES se delegan.

Casos de uso

Cada caso muestra: la situacion, **que escribes exactamente**, que hace el equipo paso a paso, y que recibes al final.

Caso 1 — Crear un proyecto nuevo desde una idea

Situacion: tienes una idea ("un sistema de notificaciones de cobranza") y quieres arrancar bien: arquitectura pensada, repos con scaffolding, backlog real.

Que escribes:

```
/dev-team:new-project sistema de notificaciones de cobranza: avisa por correo a los clientes con pagos proximos a vencer, con plantillas configurables y reportes de envio. Usuarios: analistas de cobranza. Integra con nuestro core de factoring.
```

Que hace el equipo:

1. **setup** valida tu entorno (git, Docker, gh/az, BD, Playwright) y te pide UNA confirmacion para instalar lo que falte.
2. **architect** analiza la idea y te presenta una propuesta CONCRETA:

```
Propuesta de arquitectura – Notificaciones de Cobranza
Topologia: multi-repo (3 servicios independientes + frontend)
├─ ms-notificaciones (.NET 8, Clean Architecture, MySQL) → envios y plantillas
├─ ms-programador (.NET 8, Minimal API, sin BD) → jobs de vencimientos
├─ gateway (YARP)
└─ frontend-gestion (React 18 + Vite)
BD: database-per-service · Correo: proveedor SMTP configurable
Fases: 1) plantillas+envio manual · 2) programacion automatica · 3) reportes
```

3. **Tu decides:** "prefiero mono-repo", "usa PostgreSQL", "quita el gateway" — el architect ajusta y vuelve a presentar hasta tu OK.
4. Eliges tracker (GitHub Projects o Azure DevOps Boards) y el flujo crea los repos con scaffolding completo (Dockerfile, CI/CD, tests, CLAUDE.md por servicio) desde las plantillas del plugin.
5. **product-owner** convierte la idea en HUs de negocio reales en tu tracker:

"Como analista de cobranza quiero configurar la plantilla del aviso de vencimiento para adaptar el tono al cliente" — con criterios Gherkin, story points y prioridad.

6. Se crean la **wiki** (`/wiki init`) y las metricas.

Recibes: repos listos para trabajar, backlog priorizado en tu tracker, arquitectura documentada en `architecture.md`, y el equipo configurado. Siguiendo paso típico: `/dev-team:assign-task`.

Caso 2 — Crear un proyecto desde un documento de requerimientos

Situacion: el cliente entrego un documento (Word/PDF/markdown) con los requerimientos.

Que escribes:

```
/dev-team:new-project docs/Requerimientos_Portal_Proveedores.docx
```

Diferencias con el Caso 1:

- El architect **lee el documento completo** y mapea cada requerimiento a un servicio/fase — te señala ambigüedades y vacíos ANTES de proponer ("el documento no dice si los proveedores se autentican con SSO corporativo o usuario/clave propio — ¿cual es?").
- El PO genera el backlog **trazando cada HU al requerimiento de origen** (RQ-07 → HU "Consultar estado de facturas").
- Si el documento trae pantallas o wireframes, el **ui-designer** los toma como referencia para el sistema visual (ver Caso 5).

Tip: mientras mas decisiones traigas tomadas (tracker, mono/multi, stack preferido), menos preguntas te hara el flujo. Todo lo demas lo propone el architect y tu solo apruebas o ajustas.

Caso 3 — Retomar un proyecto existente (onboard)

Situacion: un proyecto real que ya existe (varios repos, tickets abiertos, BD en uso) y quieres que el equipo lo opere desde hoy.

Que escribes (parado en la carpeta que contiene los repos, o donde quieras la carpeta paraguas):

```
/dev-team:onboard backoffice
```

Que hace el equipo:

1. **setup** valida el entorno y pregunta: ¿GitHub, Azure DevOps o solo local?
2. Detecta los repos (locales y remotos), analiza el stack y la estructura de cada uno, y **detecta la topologia** — nunca propone cambiarla.
3. Configura el acceso del **dba** a las BDs (motor, host, credenciales — se guardan en `.coordination/dba-access.json`, que JAMAS entra a git) y prueba la conexión.

4. Trae los tickets reales del tracker y arma `backlog.md` + `sprint-actual.md`.
5. Genera `repos.md` — el mapa de repos con rutas locales reales (es lo que evita que los agentes "vaguen" buscando carpetas).
6. Crea la wiki e ingiere lo detectado (arquitectura, backlog, mapa de repos).

Recibes: el equipo conoce tu proyecto y puede trabajar. Prueba con `/dev-team:status` para ver el resumen.

Tip (multi-repo): abre la sesion de Claude Code SIEMPRE desde la carpeta paraguas (donde esta `.coordination/`) y agrega los repos con `/add-dir` si estan en otra ruta — los agentes llegan a todo sin prompts de permiso.

Onboard "solo local": sin tracker remoto, el backlog vive en `.coordination/backlog.md`. Puedes conectar GitHub/Azure despues con `/dev-team:setup tracker`.

Caso 4 — Una feature de principio a fin

Situacion: los analistas necesitan filtrar las cobranzas por rango de fechas.

Que escribes:

```
/dev-team:refine los analistas necesitan filtrar las cobranzas por rango de fechas
```

Que hace el equipo (flujo completo):

0. PLAN PRIMERO — el Lead te presenta el plan y espera tu OK:

```
Plan propuesto – Filtro de fechas en cobranzas
Que:      filtro desde/hasta en el listado de cobranzas (pantalla + API)
Quien:    PO (HU) → backend (API) + frontend (pantalla) EN PARALELO
          → equipo QA valida → merge
Donde:    ms-cobranzas (branch feature/hu-42-filtro-fechas)
          frontend-gestion (branch feature/hu-42-filtro-fechas-ui)
Riesgos:  el indice actual de la tabla no cubre consultas por rango – el dba
          revisara si hace falta indice nuevo
¿Apruebas, ajustas, o lo abordamos de otra forma?
```

1. PO escribe la HU en lenguaje de negocio y la crea en el tracker:

```
# Filtrar las cobranzas por rango de fechas

**Como** analista de cobranzas
**Quiero** filtrar el listado por fecha desde/hasta
**Para** encontrar rapidamente los pagos de un periodo

## Criterios de Aceptacion
```


1. Dado un rango valido, cuando filtro, entonces veo solo cobranzas del rango
 2. Dado un rango invalido (desde > hasta), cuando filtro, entonces veo el mensaje "El rango de fechas no es valido" y el listado no cambia
 3. Dado el filtro activo, cuando limpio el filtro, entonces vuelvo al listado completo
- Story points: 3

2. **Lead** asigna: backend + frontend implementan en paralelo (branches separados, desde `origin/develop`), y QA prepara el plan de pruebas AL MISMO TIEMPO. Los devs preguntan si el entregable lleva PR antes de abrir nada.
3. **Devs terminan** → handoff a QA **con informe de conformidad** (que quedo desplegado/disponible, version, health) — sin ese informe QA no arranca.
4. **Equipo QA**: el QA Lead reparte — qa-frontend valida los criterios de pantalla (Playwright, screenshots de cada criterio) y qa-backend los de API (requests reales capturados) — en paralelo. Consolida UN veredicto: APROBADA.
5. **Lead** verifica gates (CI verde + QA aprobo + seguridad si aplica) y mergea.
6. `/dev-team:sync push` → PR mergeado, HU a Done en el tracker.
7. **tech-writer** documenta e ingiere todo a la wiki.

Recibes: la feature en `develop/main`, HU cerrada con evidencia, wiki al día.

Caso 5 — Diseñar pantallas con mockups ANTES de programar

Situacion: necesitas la pantalla nueva de "Resumen de cobranzas" y quieres elegir el diseño antes de que se escriba una linea de codigo.

Que escribes:

```
/dev-team:start necesito diseñar la pantalla de resumen de cobranzas antes de implementarla
```

Que hace el ui-designer:

1. Detecta tu sistema de diseño actual (tailwind config, tokens CSS, libreria de componentes) y captura pantallas existentes para partir de tu realidad.
2. Te entrega **2-3 propuestas de mockup** como HTML autocontenido — las abres en el browser y se ven como la pantalla real:

```
.coordination/design/resumen-cobranzas/  
├─ propuesta-A-densa.html      "orientada a datos: tabla + KPIs  
arriba"  
├─ propuesta-B-aireada.html    "orientada a lectura: cards + grafico"  
└─ design-spec.md             paleta (contraste WCAG verificado),  
tipografia,
```

espaciado, estados
(loading/error/empty/success)

3. Eliges ("la B, pero con los KPIs de la A") → el ui-designer consolida la spec final y hace handoff a **frontend** con lo que es fijo y lo que es flexible.
4. Frontend implementa EXACTAMENTE esa spec (misma paleta, mismos estados).

Recibes: pantallas decididas por ti con evidencia visual, cero retrabajo de "no era así como lo imaginaba".

Caso 6 — Reportar y corregir un bug

Situación: "la paginación del listado de cobranzas muestra registros repetidos".

Que escribes:

```
/dev-team:start bug: al pasar a la pagina 2 del listado de cobranzas
aparecen
registros que ya vi en la pagina 1
```

Que hace el equipo (ciclo completo — crear → corregir → REVALIDAR → cerrar):

1. **Lead** hace triaje (componente sospechoso, severidad) — sin implementar nada.
2. **QA reproduce** (sin debuggear): pasos exactos como usuario + screenshots numerados (00-listado-p1.png, 01-listado-p2-repetidos.png). Si no logra reproducir al primer intento o algo está caído → LEY: evidencia + **blocked** + reporte, sin insistir.
3. **PO formaliza** el bug en el tracker, en lenguaje de negocio:

"El listado de cobranzas muestra pagos repetidos al cambiar de pagina" Pasos como usuario, esperado vs obtenido, severidad e impacto, y la **evidencia embebida** (GitHub: rama **evidence** + **![] (raw)**; Azure: attachment + **** en el HTML del WI). Esto ocurre **ANTES** de hablar de quien lo corrige — aunque ya se sospeche la causa.

4. **PLAN PRIMERO:** el Lead te presenta el plan del fix (que/quien/donde/riesgos) y espera tu confirmación.
5. El **dev** corrige en branch **fix/...**; **QA escribe el test de regresión** que cubre el bug (rojo → verde).
6. **QA REVALIDA** el mismo flujo con el fix desplegado (tanda de evidencia NUEVA, subcarpeta — **revalidacion**) → veredicto APTO.
7. **PO comenta en el MISMO issue** (que se corrigió, veredicto, evidencia nueva embebida) y **te pregunta si cerrar** — nunca lo cierra solo.

Recibes: bug corregido, con historia completa y evidencia en el propio item, y un test de regresión que impide que vuelva.

Caso 7 — Debug de un problema que cruza VARIOS repos

Situacion (multi-repo): "el login funciona si pego directo al backend, pero desde la app instalada falla"
— puede ser frontend, gateway, el ms de auth o la config de infra. Nadie sabe donde esta.

Que escribes (desde la carpeta paraguas):

```
/dev-team:start el login falla desde la app desplegada (error generico),
pero
el mismo usuario funciona llamando directo al backend. Involucra frontend,
gateway y ms-auth
```

Que hace el equipo:

1. **Lead** coordina el triage transversal y te presenta el PLAN:

```
Plan de triage – Login falla solo via app
1. QA reproduce en el ambiente qa con evidencia (browser + network
log)
2. Aislamiento por capas EN PARALELO (cada agente en su repo):
  - qa-backend: request directo a ms-auth (¿200?) y via gateway (¿?)
  - infra: config del proxy/gateway y variables del contenedor
frontend
3. Con el componente identificado → PO formaliza el bug → fix dirigido
Nada se toca hasta tu OK.
```

2. **QA** reproduce UNA vez con evidencia (screenshot del error + panel de red mostrando el status real de la llamada). No insiste, no debuggea.
3. **Aislamiento en paralelo** (el Lead puede lanzar varios agentes a la vez, cada uno en SU repo):
 - qa-backend captura: directo al ms → **200 OK**; via gateway → **405**.
 - infra revisa la config real del proxy en el contenedor (no la de dev) y encuentra que el **nginx.conf** de produccion no enruta **/api**.
4. **PO formaliza** el bug apuntando al componente REAL (frontend/nginx), en lenguaje de negocio, con toda la evidencia embebida.
5. Fix dirigido por el agente dueño del repo afectado → QA revalida el flujo completo **sobre el stack real** (contenedores, no dev server) → cierre.

Claves de este flujo:

- Cada agente trabaja SOLO en su repo (un agente = un branch = un repo).
- La comparacion "directo vs via gateway" aísla la capa sin leer código.
- QA valida siempre sobre el stack real desplegado — los smokes sobre **ng serve** esconden exactamente esta clase de bug.

Caso 8 — Pruebas: plan, E2E, exploratoria y regresion

Generar el plan de pruebas de una HU:

```
/dev-team:test-plan HU-42
```

→ tabla criterio-por-criterio: cual se automatiza (E2E/API), cual es manual, casos borde adicionales y datos de prueba necesarios.

Validar una HU (interactivo + automatizado):

```
/dev-team:e2e HU-42
```

→ el equipo QA valida cada criterio con Playwright REAL (previa verificación del informe de conformidad), captura evidencia, y escribe la suite automatizada con trazabilidad criterio → test:

```
test.describe('[HU-042] Filtro de fechas en cobranzas', () => {  
  test('CA-1: filtra registros dentro del rango', async ({ page }) => { ...  
});  
  test('CA-2: muestra error con rango invalido', async ({ page }) => { ...  
});  
});
```

Correr la regresión completa (por ejemplo antes de un pase):

```
/dev-team:e2e run
```

Exploratoria libre sobre una URL:

```
/dev-team:e2e explorar http://localhost:4200
```

→ QA navega la app como usuario, reporta lo que encuentre (con evidencia), sin tocar nada.

Caso 9 — Base de datos: health, comparación de BDs y scripts de pase

Health check:

```
/dev-team:db-health full
```

→ esquema, índices no usados/redundantes, slow queries, tamaños. Si no hay conexión configurada, la pide una vez y la guarda en `dba-access.json` (fuera de git).

Comparar dos bases de datos (ej. la de qa contra la de produccion del cliente — tipico antes de un pase):

```
/dev-team:start compara la BD de qa contra la de produccion del cliente
ACME,
te paso los accesos de ambas
```

→ el **dba** compara en modo **estrictamente solo-lectura** (jamás escribe en ninguna de las dos): esquema (tablas/columnas/indices/FKs), charset/collation, data de catálogos por natural key, y volúmenes. Entrega un reporte de diferencias + scripts de nivelación GENERADOS como archivos (nunca ejecutados — los corres tu cuando decidas), en el formato global de pases.

Preparar scripts de pase — el dba entrega SIEMPRE en el formato global:

```
1_createTable.sql    2_alterTable_add.sql    3_alterTable_modify.sql
4_views.sql          5_insertInto.sql        6_procedures.sql
7_update.sql
```

100% idempotentes (re-ejecutables N veces), insert-only con guards **WHERE NOT EXISTS**, DB-agnósticos (sin **mi_db.tabla**), FKs a catálogos externos por natural key, y UTF-8 con acentos/ñ intactos (detección de mojibake incluida).

Caso 10 — Pase a un ambiente (release)

Situación: hay que pasar la versión 2.4.0 de Notificaciones de Cobranza a **Preprod PE**, con scripts de BD.

Que escribes:

```
/dev-team:pase preprod PE – Notificaciones de Cobranza v2.4.0, lleva
scripts de BD
```

Que hace el release-manager:

1. Recopila lo que falte: componentes y versiones exactas (de los repos, no inventadas), ¿que cliente si fuera productivo?, responsable (del config, jamás hardcodeado).
2. **AUDITA los scripts del DBA** — sobre el paquete RESULTANTE, checklist completo (agrupación 1-7, idempotencia, guards contados, cero **ON DUPLICATE KEY**, sin charset hardcodeado salvo **— charset-exception**: marcado, UTF-8 sin mojibake). **Si algo falla: RECHAZA** y devuelve al DBA con archivo+línea+regla — nada se consolida hasta que pase completo.
3. Consolida **Scripts.zip** (los .sql numerados en la raíz del zip).
4. Llena la **solicitud de pase** desde la plantilla oficial (secciones: componentes y versiones, temas a publicar, acciones, appsettings del ambiente DESTINO resaltados, tabla de BD, consideraciones) y exporta a PDF.

5. Entrega la **carpeta de pase completa**:

```
Release v2.4.0 16julio2026 – Notificaciones Cobranza/  
├─ Solicitud de Pase Ambientes – Preprod PE.pdf  
├─ Solicitud de Pase Ambientes – Preprod PE.docx  
└─ Scripts.zip
```

Cuando lleva documento: certificacion, puente, demo (CL/PE/CO), preprod (CO/PE) y productivos de cliente (especificando cual). Ambientes internos: solo si lo pides.

Despues del despliegue: quien despliega emite el **informe de conformidad** (sin el, QA no valida — regla de oro), y si el pase incluye validacion, el release-manager verifica antes que existan las cuentas de prueba funcionales.

Caso 11 — Auditoria de seguridad

Que escribes (en el repo del servicio, o indicando cual):

```
/dev-team:security-audit
```

Que audita cybersec (entre otros): OWASP Top 10, secretos en codigo, dependencias vulnerables, y los **checks obligatorios de auth** aprendidos de incidentes reales:

- Fallback policy GLOBAL de autorizacion (endpoints sin **[Authorize]** olvidados)
- Rate limiting APLICADO (no solo declarado) en login/MFA/reset
- Lockout que cuente tambien los fallos de MFA
- Sin fallback silencioso de autenticacion
- Flujos forzados sin bypass (cambio de contraseña obligatorio, etc.)

Recibes: reporte con hallazgos clasificados (Critico/Alto/Medio/Bajo), cada uno con su evidencia y recomendacion. **Cybersec nunca commitea** — el fix lo implementa el agente dueño del codigo, y cybersec re-audita.

Caso 12 — La wiki del proyecto

La wiki es la memoria destilada del equipo: en vez de re-leer 30 handoffs viejos, los agentes leen UNA pagina canonica al dia. Tu tambien puedes usarla.

```
/dev-team:wiki init      # una vez por proyecto (new-project/onboard ya lo  
hacen)  
/dev-team:wiki ingest    # al cierre del dia: destila handoffs nuevos a la  
wiki  
/dev-team:wiki lint      # salud: links rotos, paginas huérfanas,
```

```
desactualizadas
/dev-team:wiki query ¿por que elegimos MySQL para cobranzas?
```

La ultima responde SOLO con la wiki, citando paginas — si no sabe, lo dice.

Con Obsidian: abre [.coordination/wiki/](#) como vault → graph view del conocimiento del proyecto (HUs ↔ servicios ↔ bugs ↔ decisiones) gratis.

```
wiki/
├── servicios/ms-cobranza.md      estado vivo: stack, contratos, [[HU-042]]
├── hus/HU-042.md                historia completa con sus fuentes
├── bugs/BUG-017.md              repro + evidencia + [[fix]]
├── decisiones/ADR-003.md        por que se decidio X
├── pases/release-v2.4.0.md      que se paso y a donde
└── agentes/backend.md           memoria por rol
```

Solo el tech-writer escribe en la wiki; todos los demas la leen.

Caso 13 — Ver al equipo trabajar

Oficina virtual (en vivo, estilo Gather Town):

```
/dev-team:team-office
```

→ <http://localhost:4321>: los 15 agentes en sus salas — anillo verde girando y "escribiendo..." cuando trabajan (con su tarea debajo), rojo pulsante si estan bloqueados, sobres ✉ volando cuando hay handoff, confetti al completar. Panel lateral con feed de actividad y handoffs pendientes. Zoom con rueda, click en un agente para seguirlo. Todo local, solo lectura, cero tokens.

Metricas (para decisiones):

```
/dev-team:team-metrics      # ranking del sprint
/dev-team:team-metrics --watch # modo live
```

→ por agente: tareas completadas, handoffs, commits, lead time real, tokens y costo estimado segun su modelo — con alertas accionables ("cybersec consume 18% de tokens con 4% de las tareas → considera bajarlo de modelo").

Estado en texto:

```
/dev-team:status
```

Caso 14 — Trabajar un sprint completo con Scrum

1. `/dev-team:refine {pedidos del sprint}` → P0 crea/refina HUs (INVEST, story points)
2. Sprint Planning: el P0 propone el Sprint Goal y las HUs candidatas; el Lead valida capacidad → `sprint-actual.md` queda como tablero
3. `/dev-team:assign-task` → plan primero + asignaciones
4. Durante el sprint:
 - `/dev-team:status` cada mañana (tu "daily")
 - `/dev-team:sync` para reflejar avances en el tracker
 - `/dev-team:team-office` si quieres verlo en vivo (nada entra al sprint en curso sin tu decision explicita)
5. Sprint Review: el P0 verifica HU por HU contra criterios y el Sprint Goal; lo no terminado VUELVE al backlog (no se arrastra en silencio)
6. Retrospectiva: acuerdos de mejora → handoff al Lead; los de redaccion los aplica el P0 desde el siguiente sprint
7. `/dev-team:wiki ingest` → el sprint queda en la memoria del equipo

Caso 15 — Ajustar los modelos de los agentes

Por proyecto — `.coordination/config.json`:

```
"team": { "models": { "architect": "opus" } }
```

Por usuario (todos TUS proyectos) — `~/.claude/dev-team.config.json`:

```
{ "team": { "models": { "architect": "opus" } } }
```

Orden de resolucion: **proyecto** → **personal** → **default del agente**. Valores: `haiku` | `sonnet` | `opus` (`fable` prohibido). **Fijos e inalterables**: `setup` y `tech-writer` (`haiku`) — cualquier entrada para ellos se ignora.

Casos tipicos:

- Proyecto complejo → `"architect": "opus"`
- Plan con limites ajustados → deja todo en default (ya es el minimo sano)

Configuracion del proyecto

Todo vive en `.coordination/config.json` (lo crean `new-project/onboard`):


```
{
  "project": "backoffice",
  "topology": "multi",
  "tracker": {
    "provider": "azure",
    "azure": { "org": "{org}", "project": "{proyecto}" },
    "reviewer": "{email-del-reviewer}",
    "overheadEpicId": 1234,
    "areaPath": "{Proyecto}\\{Equipo}",
    "iterationPath": "{Proyecto}\\Sprint {N}"
  },
  "git": {
    "defaultBranch": "develop",
    "identity": { "name": "{Nombre Apellido}", "email": "{email-del-proyecto}" }
  },
  "pase": {
    "templatePath": "(opcional – default: plantilla del plugin)",
    "outputDir": "(opcional – default: .coordination/pases/)",
    "elaboradoPor": "{Nombre Apellido}"
  },
  "team": {
    "models": { "architect": "opus" }
  },
  "urls": { "dev": "http://localhost:3000" }
}
```

Campo	Efecto
<code>topology</code>	<code>mono/multi</code> : como trabajan los agentes y donde vive <code>.coordination/</code>
<code>tracker.provider</code>	<code>github/azure</code> : donde viven HUs, items y PRs
<code>tracker.reviewer</code>	Reviewer que los devs ponen en cada PR
<code>tracker.overheadEpicId</code>	Epica/PBI padre para fixes sueltos sin epica propia
<code>git.defaultBranch</code>	Rama base OBLIGATORIA para ramificar (via <code>git fetch origin</code>)
<code>git.identity</code>	Identidad de commits del proyecto (nunca la default del agente)
<code>pase.*</code>	Plantilla, carpeta de salida y "elaborado por" de las solicitudes
<code>team.models.{agente}</code>	Override de modelo en ESTE proyecto (no aplica a setup/tech-writer)
<code>urls.dev</code>	URL del ambiente que QA usa para validar

Estructura completa de `.coordination/`:

```
.coordination/
├─ config.json      fuente de verdad (arriba)
```

```

|— handoffs/ (+archive/) comunicacion entre agentes
|— wiki/                wiki viva (vault de Obsidian)
|— metrics/             activity.jsonl (eventos, via hooks)
|— evidence/            screenshots/clips de QA
|— pases/               carpetas de pase entregadas
|— office/              la oficina virtual (se instala con /team-office)
|— test-plans/          planes de prueba de QA
|— backlog.md · sprint-actual.md · architecture.md · repos.md
|— dba-access.json      credenciales BD – NUNCA en git
|— setup-status.json    estado del entorno – NUNCA en git

```

Como cuidar tu limite de uso

Revisa tu panel con `/usage`. Los habitos que mas ahorran:

1. **Sesiones cortas por tarea** — el enemigo #1 es el contexto gigante: una sesion de 3 horas re-lee 150k+ tokens en CADA turno. `/compact` a mitad de tarea larga, `/clear` (o sesion nueva) al cambiar de tema.
2. **Wiki al dia** — `/dev-team:wiki ingest` al cierre hace que mañana la sesion arranque liviana (pagina canonica de 2-3k tokens vs historial completo).
3. **Pregunta directa = respuesta directa** — sin `/start` para preguntas simples.
4. **Modelo principal en sonnet** (`/model sonnet`).
5. **MCP solo los necesarios** — `claude mcp list` y quita los que no uses.

Protecciones automaticas del plugin: los subagentes no pueden delegar (solo el lead), los comandos corren inline (nunca como subagentes), el logging de actividad va por hooks (cero tokens), y los agentes usan contexto bajo demanda (si el handoff trae todo, trabajan de inmediato sin re-leer).

Leyendo tu `/usage`:

- "subagents under dev-team:backend" bajo = normal (Explores permitidos); alto = version vieja del plugin → actualiza y reinicia
- "% at >150k context" alto = sesiones demasiado largas → habito 1
- La primera interaccion tras el reset siempre pesa mas (cache fria) — es el peaje de arranque, no una fuga

Solucion de problemas

"Failed to update plugin: Plugin dev-team not found" → usa el nombre completo: `claude plugin update dev-team@faast-marketplace`.

Actualice el plugin pero los agentes siguen igual → los agentes/hooks se cargan al INICIO de sesion: cierra y abre la sesion. Verifica la version con `claude plugin list`.

La oficina virtual se ve vacia / nadie se mueve → (1) ¿reiniciaste la sesion tras actualizar? Los hooks que registran actividad se cargan al inicio. (2) ¿el server apunta al `.coordination` correcto? (`node .coordination/office/server.mjs --dir .coordination`). (3) `python3 --version` — los hooks lo necesitan. La oficina se llena a medida que los agentes trabajan.

Un agente quiso crear otro agente y fue bloqueado → correcto, es el diseño: solo el Lead delega. El agente debe hacer su trabajo directo o dejar handoff al Lead.

QA no quiere validar → tambien correcto si falta el informe de conformidad o el stack completo en desa (regla de oro, inalterable). Pide a quien desplego que emita el informe.

Los agentes tardan en arrancar → (1) sesion abierta desde la carpeta paraguas + `/add-dir` para los repos; (2) `repos.md` con las rutas locales reales; (3) wiki al dia — el arranque lee 1 pagina en vez de 5 archivos.

El architect no uso opus aunque lo configure → el override lo aplica quien INVOCA (lead/flujos) leyendo el config: verifica la clave exacta `team.models.architect` en `.coordination/config.json` (proyecto) o `~/claude/dev-team.config.json` (personal), y que la sesion sea nueva.

Un pase fue rechazado por el release-manager → es el gate funcionando. El handoff de rechazo lista archivo+linea+regla; lo corrige el DBA (o infra si el problema es el consolidador) y se re-audita. Nunca se "arregla" editando scripts ya aplicados.

Preguntas frecuentes

¿Tengo que saber que agente hace cada cosa? No. `/dev-team:start` + describir lo que necesitas. El equipo enruta.

¿Puedo confiar en que no haran nada sin avisarme? Si: plan primero es regla — features y fixes te presentan el plan y esperan tu OK. Puedes ajustar o pedir otro abordaje. Solo lo de lectura corre directo.

¿Funciona sin GitHub ni Azure DevOps? Si — "solo local" en el onboard; el backlog vive en `.coordination/backlog.md`. Conecta un tracker despues con `/dev-team:setup tracker`.

¿Por que QA no arregla los bugs que encuentra? Por diseño: QA reproduce y reporta con evidencia; el dev corrige. Asi la evidencia es objetiva y el fix lo hace quien conoce el codigo.

¿Que pasa si QA se topa con algo caido o que no funciona a la primera? LEY: captura la evidencia de ese primer intento, registra `blocked` y reporta de inmediato. No reintenta, no busca workarounds, no toca nada.

¿Donde queda la evidencia de QA? Local en `.coordination/evidence/` mientras valida; al formalizar, EMBEBIDA en el item del tracker (GitHub: rama `evidence + ![] (raw)`; Azure: attachment + `<img>` en el HTML del WI). Siempre visible dentro del item, jamas un link suelto.

¿El DBA puede escribir en las BDs cuando compara dos bases? No. Solo-lectura por regla dura. Los scripts de nivelacion se GENERAN como archivos que tu decides cuando ejecutar.

¿Siempre se crea un PR al terminar una tarea? No. Los devs preguntan primero — hay entregables que no llevan PR.

¿Necesito Obsidian para la wiki? No — es markdown puro y los agentes la usan igual. Obsidian agrega el graph view y navegacion comoda para humanos.

¿Puedo usar solo una parte del equipo? Si. Cada comando funciona independiente: solo `/db-health`, solo `/refine`, solo `/pase`, solo `/e2e`.

¿El equipo respeta mi proyecto tal como esta? Si. En onboard nunca se propone cambiar topologia, stack ni convenciones, salvo que lo pidas.

¿Como cambio de GitHub a Azure DevOps (o al reves)? Edita `tracker.provider` en `.coordination/config.json` y corre `/dev-team:setup tracker` para validar la autenticacion del nuevo proveedor.